Author

author: name: Ромеро Гаспар Фернандо Алексей degrees: DSC orcid: 0009-0004-8816-345X email: 1132255520@rudn.ru affilition: - name: Рщссийский университет дружбы народов country: Российская Федерфция postal-code: 117198 city: Москва address: ул. Миклухо-Маклая д. 21к3

Title

title: “Отчёт по лабораторной работе № 2” subtitle: “Настройка Git, ключи SSH и PGP” license: “CC BY” abstract: | В данном отчёте представлены результаты выполнения лабораторной работы по теме: базовая настройка Git, создание ключей SSH и PGP для безопасной работы с GitHub.

Описаны установка и настройка Git, генерация ключей SSH (Ed25519 и RSA), создание PGP-ключей с помощью GPG, добавление их в учётную запись GitHub, а также настройка автоматической подписи коммитов.

Сделаны выводы о достижении поставленной цели и о важности безопасной аутентификации при работе с удалёнными репозиториями. keywords: [Git, SSH, PGP, GPG, GitHub, аутентификация, подпись коммитов, безопасность] —

##Введение

1. Цель работы
2. Теоретические сведения > 2.1. Системы контроля версий. Общие понятия > 2.2. Примеры использования git > 2.3. Основные команды git > 2.4. Стандартные процедуры работы при наличии центрального репозитория
3. Базовая настройка git > 3.1. Учёт переносов строк
4. Создание ключа SSH > 4.1. Общая информация
5. Верификация коммитов с помощью PGP > 5.1. Обжая информация
6. Проверка коммитов в GIT > 6.1. Режим бдительности (vigilant mode)
7. Задание и сопровождение работ > 7.1. Установка программного обеспечения > 7.1.1. Установка git > 7.1.2. Установка gh > 7.2. Базовая установка git > 7.3. Создание ключа SSH > 7.4. Создание ключей PGP > 7.5. Установка GitHub > 7.6. Добавление PGP-ключа на Github > 7.7. Настройка автоматической подписи коммитов в Git > 7.8. Настройка gh > 7.9. Шаблон для рабочего пространства > 7.9.1.

Создание репозитория курса на основе шаблона > 7.9.2. Настрой каталога курса
8. Контрольные вопросы
9. Выводы
10. Список литературы

Цель работы

Целью данной лабораторной работы является изучение идеологии и применения систем контроля версий,а также освоение базовых навыков работы с распределённой системой контроля версий Git. В ходе работы необходимо выполнить первоначальную настройку Git,создать SSH-и PGP-ключи для обеспечения безопасной работы с удалёнными рапозиториями,настроить автоматическую подпись коммитов,а также зарегистрироваться на платформе GitHub и создать локальный каталог для выполнения заданий по курсу.

Теоретические сведения

В данном разделе представлены основные концепции систем контроля версий (VCS) и Git, необходимые для процесса разработки на практике.

2.1. Системы контроля версий. Общие понятия

Система контроля версий (VCS)-это программное обеспечение,преднасченное для управления и упорядочивания различных правок и измений в файлах проекта,особенно в исходном коде. Ее ключева функция-ведение истории правок,что позволяет разработчикам восстанавливать предыдущие версии кода, эффективно сотрудничать в команде и поддерживать организованный рабочий процесс.

Ключевые преимущества венчурных капиталистов:

- Полная история: Позволяет просматривать и откатываться к любой предыдущей версии проекта.

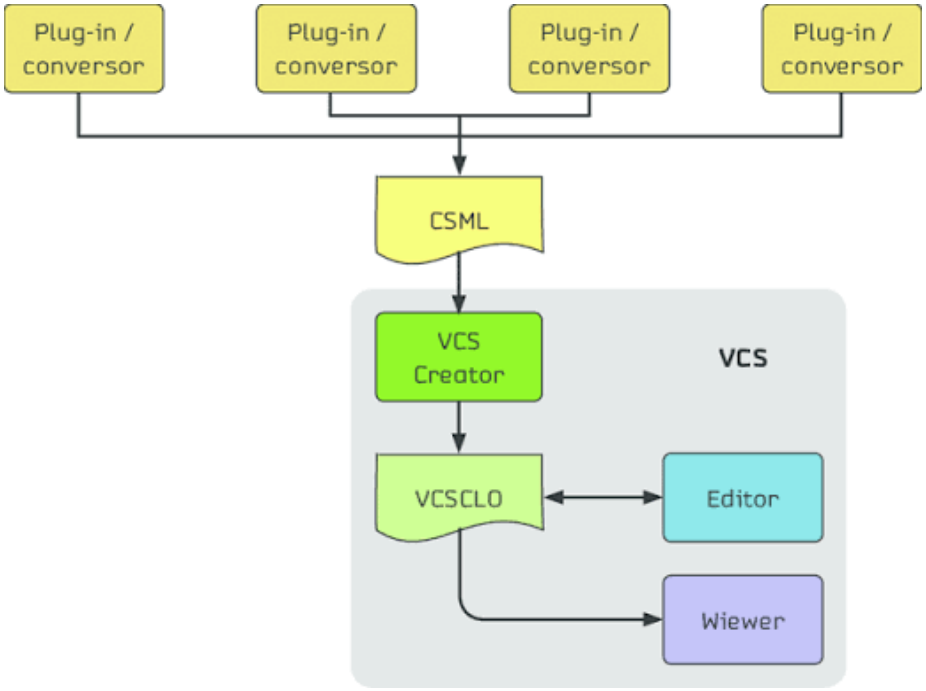

Plug-in / conversor
Plug-in / conversor
Plug-in / conversor
Plug-in / conversor
CSML
VCS Creator
VCS
VCSCLO
Editor
Wiewer

Рис. 1: VCS

- Совместная работа: Упрощает одновременную работу нескольких разработчиков над одним и тем же проектом без измений.
- Безпасность: Помогает предотвратить потерю кода и ведет учет того,кто вносил каждое изменение.

Венчурные капиталисты в основном делятся на два типа:

Централизванные системы(CVCS): В этой модели существует единый центральный репозиторий,расположенный нв сервере,в котором хранятся все версии проекта. Разработчики загружают рабочую копию файдов с этого центрального сервера и после внессения изменений заружают их обратно в центральный репозиторий. Примерами систем такого являются CVS и Subversion(SVN)

Распределенные системы контроля версий(DVCS): В DVCS каждый разработчик получает полную локальную копию рапосзитория,включающую весь журнал версий. Это позволяет отказаться от централизованного сервера,поскольку каждая локальная копия сама по себе служит рапозиторием. Эта система позволяет работать автономно, без подключения к интернету. Примерами могуть служить Git b Mercurial.

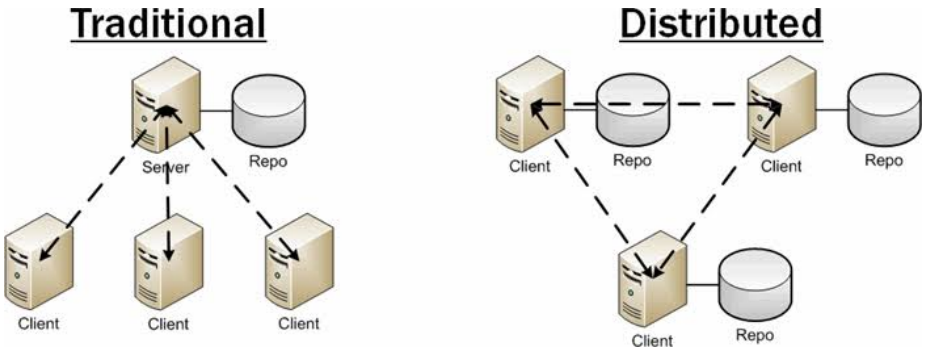


Рис. 2: CVCS and DVCS

2.2. Примеры использования git

Git-это распределенная система контроля версий, первоначально созданная Линусом Торвальдом для разработки ядра Linux. Его популярность заключается в его эффективности,гибкости и мощной системе ответлений.

Преимущества Git:

- Распределенная модель:У каждого разработчика есть полная копия рапозитория,что позволяет работать без постоянного подключения к Интернету.
- Гибкие ветви(branches):Посволяют легко создавать незавимые линии разработки, облегчая параллельное экспериментирование и работу.
- Производительность:Git работает чрезвычайно быстро и эффективно даже в больших проектах.
- Целостность: Каждое изменение идентифицируется с помощью криптографического хэша,который обеспечивает целостность истории.

2.3. Основные команды git

Ниже перечислены наиболее часто используемые команды в Git:

- git init:Создает новый локальный репозиторий в текущем каталоге.
- git add:Добавляет файлы в промежуточную область (промежуточную область) для следующей фиксации.
- git commit:Сохраняет подготовленные изменения в локальном рапозитории с описательным сообщением.
- git status:Отображает состояние файлов в репозитории.
- git diff:Показывает различия между текущим состоянией и последней фиксацией.
- git pull:Извлекает измения из удаленного репозитория и объединяет их с локальным заданием.
- git push:Отправляет локальные измения в удаленный рапозиторий.
- git branch:Список,создание или удаление ветвей.
- git checkout:Переключение между ветвями или восстановление файлов.
- git merge:Объединяет изменения из одной ветки в другую.
- git clone:Создает локльную копию удаленного рапозитория.

2.4. Стандартные процедуры работы при наличии центрального репозитория

При работе с центральным репозиторием типичный рабочий процесс включает следующие этапы:

1. Проверка и получение обновлений из центрального рапозитория:git checkout master,git pull,git chekout -b имя_ветки.

2. Внесение изменений в код:редактирование необходимых файлов.
3. Проверьте статус изменений:статус git.
4. Просмотрите различия:разница в git.
5. Добавьте изменения в промежуточную область:мерзавец добавить.
6. Сохраните изменения с помощью описательного сообщения:git commit-am “Сообщение о фиксации”.
7. Отправьте изменения в центральный репозиторий:git push origin имя_файла_файла

Это процедура гарантирут,что изменения будут правильно интегрированы и сохранена чистая и организованная история.

Базовая настройка git

3.1. Учёт переносов строк

Операционные системы используют разные символы для обозначения конца строки в текстовых файлах:

- Windows:\r\n (CRLF)
- Unix/Linux/macOS:\n (LF)

Git позволяет управлять этими различиями с помощью параметров конфигурации,которые нормализуют разрывы строк в репозитории,предотвращая конфликты при совместном использовании кода между различными системами:

Параметр core.autocrlf:

этот параметр определяет,как git обрабатывает окончания строк при выполнении коммитов и проверок:

- core. autocrlf true (Windows):Преобразует CRLF в LF при фиксации и LF в CRLF при оформлении заказа,адаптируя файлы к операционной системе.
- core. autocrlf (Linux/macOS):Преобразует CRLF в LF при фиксации,но не изменяет файлы при оформлении заказа.

Чтобы настроить его:

- Для систем Windows:git config –global core. autocrlf true

- Для систем Linux/macOs:git config –глобальный ввод core. autocrlf

Параметр core. safe crlf:

Этот параметр позволяет проверить,является ли преобразование кщнца строки обратимым,предотвращая повреждение двоичных файлов:

- core. safecrlf true:Отлоняет коммиты с необратимыми преобразованиями.
- core. safecrlf warn:Показывает предупреждение,но разрешает фиксацию.

Создание ключа SSH

4.1. Общая информация

SSH (Secure Shell) — это сетевой протокол, обеспечивающий безопасное соединение между двумя системами через незащищенную сеть с помощью криптографической аутентификации. В контексте Git SSH-ключи применяются для авторизации пользователя на удаленных серверах, таких как GitHub, GitLab или Bitbucket, что позволяет выполнять операции вроде git push или git pull без необходимости вводить учетные данные при каждом действии.

Система аутентификации SSH основана на асимметричной паре ключей:

- Закрытый ключ-Остается на компьютере пользователя и никогда не должен передаваться другим лицам. Это ключ, который позволяет аутентифицировать личность пользователя.
- Открытый ключ-копируется на удаленный сервер (например, в учетную запись GitHub).
- Сервер использует его для проверки подлинности закрытого ключа.

Алгоритмы шифрования SSH:

SSH поддерживает несколько алгоритмов генерации ключей,наиболее рекомендуемых в настоящее время :

- Ed25519:Считается самым безопасным и эффективным.Он рекомендуется по умолчанию из-за его устойчивости к атакам и небольшого размера.
- RSA:широко применяемый алгоритм,однако для обеспечения безопасности рекомендуется использовать ключ длиной не менее 4096 бит.

Расположение ключей:

По умолчанию ключи SSH хранятся в каталоге~/. ssh/. Типичные имена:

- Закрытые ключи: id_rsa, id_ed25519
- Открытые ключи: id_rsa.pub, id_ed25519. pub

Верификация коммитов с помощью PGP

5.1. Общая информация

PGP (Pretty Good Privacy) — это криптографический инструмент, который позволяет подписывать коммиты цифровой подписью, гарантируя их подлинность и целостность. Когда коммит подписан, Git добавляет к нему цифровую подпись, созданную с использованием приватного ключа пользователя. Любой, у кого есть публичный ключ,может проверить, что коммит действительно был создан указанным автором и не был изменён.

Подписание коммитов с помощью PGP обеспечивает дополнительный уровень безопасности и доверия к программным проектам . Когда коммит подписан, Git прикрепляет цифровую подпись, используя закрытый ключ пользователя. Любой, у кого есть открытый ключ, может убедиться, что коммит является подлинным и не был изменен.

Преимущества подписания коммитов:

- Подлинность:Доказывает,что коммит был создан законным автором,предотвращая возможность того, чтобы кто-либо выдавал себя за разработчика.
- Целостность:Гарантирует,что код не был изменен с момента фиксации.
- Доверие:Для сопровождающих проектов подписанные коммиты обеспечивают уверенность в том,что код получен из надежного источника.

В GitHub коммиты, подписанные действительным PGP-ключом, помечаются зелёной галочкой «Verified».

Проверка коммитов в Git

В этом разделе рассказывается, как GitHub проверяет подлинность подписанных коммитов и как пользователи могут отслеживать состояние подписей в своих репозиториях,а также о дополнительных мерах безопасности, предоставляемых платформой.

6.1. Режим бдительности (vigilant mode)

GitHub предлагает функцию безопасности под названием «Vigilant Mode» («Режим бдительности»), которая позволяет пользователям и организациям настраивать более строгий уровень проверки подписей.

Что же делает этот режим?

Давайте разберемся.· Все неподписанные коммиты помечаются как «Непроверенные»: при включённой этой опции любой неподписанный коммит или коммит с неподтверждённой подписью будет явно помечен как «Непроверенный» на GitHub.

Повышенная безопасность и прозрачность: эта настройка особенно ценна для проектов, где требуется высокий уровень защиты.

Она обязывает участников подписывать свои коммиты и позволяет легко выявлять тех, кто не соблюдает политику подписания.

Как включить режим Vigilant Mode:

1. Авторизуйтесь в GitHub.
2. Перейдите в Настройки (Настройки) → Ключи SSH и GPG.
3. В разделе “Ключи GPG” установите флажок “Отмечать неподписанные коммиты как непроверенные” (Flag unsigned commits as unverified).

Эффекты активации режима бдительности:

Тип фиксации	Поведение без бдительного режима	Поведение с бдительным режимом
Подписанный коммит с действительным ключом	Проверено	Проверено
Подписанный коммит с несвязанным ключом	Непроверенный	Непроверенный
Зафиксировано без подписи	Без метки (ничего не отображается)	Непроверенный

Задание и сопровождение работ

В этоь разделе предстаилены задвчи,которые необходимо выполнить,и подробные инструкции по выполнению начальной настройки Git,генерации ключей SSH и PGP,связыванию с GitHub и созданию репозитория для курсовых работ.

7.1. Установка программного обеспечения

Для работы с Git и GitHub из терминала необходимо установить два основных пакета:Git (система контроля версий) и GitHub CLI (интерфейс командной строки для взаимодействия с GitHub).

7.1.1. Установка git

```
liveuser@localhost-live:~$ sudo dnf install git -y
Updating and loading repositories:
 Fedora 44 - x86_64                      100% |  24.2 KiB/s |  25.4 KiB |  00m00s
 Fedora 44 - x86_64 - Updates            100% |   1.8 MiB/s |  10.9 MiB |  00m06s
 Fedora 44 openh264 (From Cisco) - x86_ 100% | 936.0   B/s |   5.3 KiB |  00m06s
 Fedora 44 - x86_64                      100% |   3.6 MiB/s |  38.3 MiB |  00m11s
Repositories loaded.
Package "git-2.53.0-1.fc44.x86_64" is already installed.

Nothing to do.
```

Рис. 3: Install git

Опция -y автоматически подтверждает установку без необходимости ручного вмешательства.

Проверка установки:

Для проверки правильности установки Git была выполнена следующая команда:

```
liveuser@localhost-live:~$ git --version
git version 2.53.0
```

Рис. 4: watch git version

Ожидаемый результат должен отображать установленную версию.

7.1.2. Установка gh

Для установки GitHub CLI была выполнена следующая команда:

```
liveuser@localhost-live:~$ sudo dnf install gh -y
Updating and loading repositories:
Repositories loaded.
Package                 Arch    Version                 Repository        Size
Installing:
 gh                     x86_64  0:2.97.0-2.fc44         updates       39.9 MiB

Transaction Summary:
 Installing:         1 package

Total size of inbound packages is 12 MiB. Need to download 12 MiB.
After this operation, 40 MiB extra will be used (install 40 MiB, remove 0 B).
[1/1] gh-0:2.97.0-2.fc44.x86_64         100% |   7.5 MiB/s |  11.8 MiB |  00m02s
--------------------------------------------------------------------------------
[1/1] Total                             100% |   5.1 MiB/s |  11.8 MiB |  00m02s
Running transaction
Importing OpenPGP key 0x6D9F90A6:
 UserID     : "Fedora (44) <fedora-44-primary@fedoraproject.org>"
 Fingerprint: 36F612DCF27F7D1A48A835E4DBFCF71C6D9F90A6
 From       : file:///etc/pki/rpm-gpg/RPM-GPG-KEY-fedora-44-x86_64
The key was successfully imported.
[1/3] Verify package files              100% |   8.0   B/s |   1.0   B |  00m00s
[2/3] Prepare transaction               100% |   2.0   B/s |   1.0   B |  00m00s
[3/3] Installing gh-0:2.97.0-2.fc44.x86 100% | 180.1 MiB/s |  40.0 MiB |  00m00s
```

Рис. 5: install gh

Проверка установки:

Для проверки правильности установки GitHub CLI была выполнена следующая команда:

```
liveuser@localhost-live:~$ gh --version
gh version 2.97.0 (2026-07-31)
https://github.com/cli/cli/releases/tag/v2.97.0
```

Рис. 6: watch gh version

Ожидаемый результат должен отображать установленную версию.

7.2. Выполнение базовой настройки git

Начальная настройка Git необходима для идентификации автора коммита и подготовки рабочей среды. Эта настройка выполняется глобально и применяется ко всем репозиториям пользователя.

Параметры для настройки:

- user.name:Определяет имя пользователя,которое будет отображаться в каждом коммите.
- user.mail:Определяет адрес электронной почты,связанный с коммитами. Он должен совпадать с адресом электронной почты,зарегистрированным в GitHub,для корректной проверки подписей PGP.
- core.quitepath:Настраивает кодировку UTF-8,чтобы избежать проблем со специальными символами в сообщениях коммитов.
- init.defaultBranch:Устанавливает имя ветки по умолчанию при создании нового репозитория (в данном случае,vaster).

```
liveuser@localhost-live:~$ git config --global user.name "Fernando Alexei Romero
 Gaspar"
liveuser@localhost-live:~$ git config --global user.mail "1132255520@rudn.ru"
liveuser@localhost-live:~$ git config --global core.quotepath false
liveuser@localhost-live:~$ git config --global init.defaultBranch master
liveuser@localhost-live:~$ git config --global core.autocrlf input
liveuser@localhost-live:~$ git config --global core.safecrlf warn
```

Рис. 7: use of config global

Для проверки правильности конфигурации была выполнена команда:

В результате настройка была применена успешно. Все параметры отображаются в выводе команды.

```
liveuser@localhost-live:~$ git config --list
user.name=Fernando Alexei Romero Gaspar
user.mail=1132255520@rudn.ru
core.quotepath=false
core.autocrlf=input
core.safecrlf=warn
init.defaultbranch=master
```

Рис. 8: list of the git config

7.3. Создание ключей SSH

Аутентификация SSH основана на паре асимметричных ключей:

- Приватный ключ:Чранится на локальной машине пользователя и никогда не должен передаваться третьим лицам.
- Публичный ключ:Загружается на удалённый сервер (например,в GitHub) и используется для проверки подлинности приватного ключа.

Для выполнения лавораторной работы были сгенерированы ключи по двум алгоритмам: Ed25519 (рекомендуемый) и RSA (как фльтернативный вариант для совместимости).

Былы выполнены следующие команды:

```
liveuser@localhost-live:~$ ssh-keygen -t ed25519 -C "1132255520@rudn.ru"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/liveuser/.ssh/id_ed25519):
Created directory '/home/liveuser/.ssh'.
Enter passphrase for "/home/liveuser/.ssh/id_ed25519" (empty for no passphrase):

Enter same passphrase again:
Your identification has been saved in /home/liveuser/.ssh/id_ed25519
Your public key has been saved in /home/liveuser/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:In+/Vq5Wbrr57r+xPUbiqAUNbXzxH2IGpGxdoeug73o 1132255520@rudn.ru
The key's randomart image is:
+--[ED25519 256]--+
|          .o +.  |
|        . = + o  |
|         = * = o |
|        . + = . o|
|    . . So o    .|
|     o .. + o. . |
|      ...  Bo +  |
|       ..E+o+. * |
|       .+*OX+.=.o|
+----[SHA256]-----+
```

```
liveuser@localhost-live:~$ ssh-keygen -t rsa -b 4096 -C "1132255520@rudn.ru"
Generating public/private rsa key pair.
Enter file in which to save the key (/home/liveuser/.ssh/id_rsa):
Enter passphrase for "/home/liveuser/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/liveuser/.ssh/id_rsa
Your public key has been saved in /home/liveuser/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:YWVJo5mWFMXb8uOOIY679huUvD5QBRRi3AXFDGxe2FI 1132255520@rudn.ru
The key's randomart image is:
+---[RSA 4096]----+
|     .o+O^E.     |
|     ..o*BO.     |
|       oO+ o     |
|       +ooo .    |
|       .S  o     |
|      .. .  o    |
|       .+ .. .   |
|      .+.o o.    |
|     .++=o...    |
+----[SHA256]-----+
```

В процессе генерации было предложено указать имя файла для сохранения ключа (оставлено по умолчанию) и задать парольную фразу (passphrase) для дополнительной защиты приватного ключа.

Для проверки создания ключей была выполнена команда.

В результате в каталоге ~/.ssh/ появились файлы:

- id_ed25519:Приватный ключ Ed25519
- id_ed25519.pub:Публичный ключ Ed25519
- id_rsa:Приватный ключ RSA
- id_rsa.pub:Публичный ключ RSA

7.4. Создание ключей PGP

Для создания новой пары ключей была выполнена команда:

```
liveuser@localhost-live:~$ gpg --full-generate-key
gpg (GnuPG) 2.4.9; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

gpg: directory '/home/liveuser/.gnupg' created
Please select what kind of key you want:
   (1) RSA and RSA
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
   (9) ECC (sign and encrypt) *default*
  (10) ECC (sign only)
  (14) Existing key from card
Your selection? 1
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072) 4096
Requested keysize is 4096 bits
Please specify how long the key should be valid.
         0 = key does not expire
      <n>  = key expires in n days
      <n>w = key expires in n weeks
      <n>m = key expires in n months
      <n>y = key expires in n years
Key is valid for? (0) 0
Key does not expire at all
Is this correct? (y/N) y

GnuPG needs to construct a user ID to identify your key.

Real name: Fernando Alexei Romero Gaspar
Email address: 1132255520@rudn.ru
Comment: lab2
You selected this USER-ID:
    "Fernando Alexei Romero Gaspar (lab2) <1132255520@rudn.ru>"

Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? 0
Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? o
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: /home/liveuser/.gnupg/trustdb.gpg: trustdb created
gpg: directory '/home/liveuser/.gnupg/openpgp-revocs.d' created
```

```
gpg: revocation certificate stored as '/home/liveuser/.gnupg/openpgp-revocs.d/1F
3A99CF8CB5FFAD018629DAFF488D4DD8B73CF6.rev'
public and secret key created and signed.

pub   rsa4096 2026-08-08 [SC]
      1F3A99CF8CB5FFAD018629DAFF488D4DD8B73CF6
uid                      Fernando Alexei Romero Gaspar (lab2) <1132255520@rudn.r
u>
sub   rsa4096 2026-08-08 [E]
```

В процессе генерации были выбраны следующие параметры:

- Тип ключа:RSA and Rsa(рекомендуемый)
- Размер ключа:4096 бит
- Срок действия:0 (не истекает)
- Имя пользоваьеля:Fernando Alexei Romero Gaspar
- Электронная почта:1132255520@rudn.ru
- Комментарий:(остовлен пустым)

Проверка создания ключа и получение идентифмкатора:

Для просмотра созданных ключей и получения идентификатора (Key ID) была выполнена команда:

```
liveuser@localhost-live:~$ gpg --list-secret-keys --keyid-format LONG
gpg: checking the trustdb
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: depth: 0  valid:   1  signed:   0  trust: 0-, 0q, 0n, 0m, 0f, 1u
[keyboxd]
```

Рис. 9: list of the GPG keys

В выводе команды была получена следующая информация:

```
---------
sec   rsa4096/FF488D4DD8B73CF6 2026-08-08 [SC]
      1F3A99CF8CB5FFAD018629DAFF488D4DD8B73CF6
uid                 [ultimate] Fernando Alexei Romero Gaspar (lab2) <1132255520@
rudn.ru>
ssb   rsa4096/A0F26A77556505A7 2026-08-08 [E]
```

Рис. 10: Result of the comand

7.5. Настройка GitHub

Для взаимодействия с GitHub из терминала и безопасного выполнения таких операций,как ‘git push’ и ‘git pull’,необходимо

добавить открытый SSH-ключ, сгенерированный на предыдущем шаге,в вашу учетную запись GitHub. Это позволить GitHub распознать ваш компьютер как авторизованное устойство.

1. Скопируйте открытый ключ в буфер обмена:

Для получения содержимого открытого ключа и копирования его в буфер обмена использовалсь следующая команда:

```
liveuser@localhost-live:~$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIBPhez9afr6Pk8rHCDa9tkj7hC6sRD0MYbmYlkdn6c8M
 1132255520@rudn.ru
```

Рис. 11: comad ssh

2. Добавление ключа в GitHub:

Затем были выполнены следующие шаги в веб-интерфейсу GitHub:

- Войдите в GitHub
- Перейдите в Настройки→Ключи SSH и GPG.
- Нажмите кнопку “Новый ключи SSH”.
- В поле “Заголовок” введите описательное имя:Fedora Laptop.
- В поле “Ключ” вставьте ранее скопированный открытый ключ.
- Нажмите “Добавить ключ SSH”,чтобы сохранить ключ.

Проверка SSH-соединения:

Чтобы убедиться,что ключ добавлен правильно и соединеник с GitHub работает,выполните следующую команду:

Это сообщеник подтверждает,что аутентификация SSHпрошла успешно и что машина авторизована для взаимодействия с удаленным рапозиторием.

7.6. Добавление PGP-ключа в GitHub

Чтобы GitHub мог проверить подлинность коммитов,подписанных нашим PGP-ключом,необходимо добавить открытый ключ в учетную запичь GitHub. После добавления все коммиты,подписанные соответствующим закрытым ключом,будут отображать значок “Проверено” на GitHub.

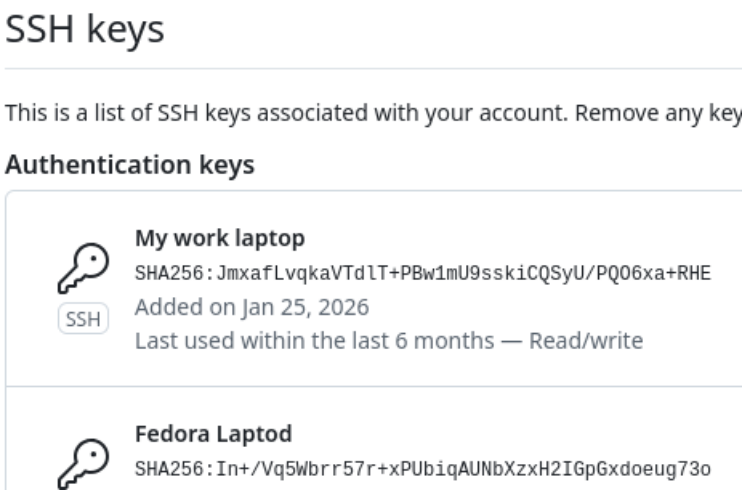


Рис. 12: Key SSH in GitHub

```
liveuser@localhost-live:~$ ssh -T git@github.com
The authenticity of host 'github.com (140.82.121.3)' can't be established.
ED25519 key fingerprint is: SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
Hi AlexeiFRG1! You've successfully authenticated, but GitHub does not provide sh
ell access.
```

Рис. 13: verification key SSH

Добавление PGP-ключа в GitHub:

Чтобы получить открытый ключ в формате, требуемом GitHub,ключ был экспортирован с использованием идентификатора (Key ID),полученного на предыдущем шаге:

```
liveuser@localhost-live:~$ gpg --armor --export FF488D4DD8B73CF6
-----BEGIN PGP PUBLIC KEY BLOCK-----

mQINBGp3CYsBEADmdK8kKi/lJgomIiPDcZmrI/VWwFrRprwbFJDvztefHo0qL/Bb
SOdyXJksCZA9m4YUCTC4HsFqiNJMIOF3xfia+6KgzCpkN4kSJTdFaAWLYrt7koTZ
eVp3HF/+5qF7r1FUM31s0BT4vv4xlY1BbXAlT9HEudpRJAi6XYEmvmCK8d9MxI/G
iJlFhm2yRauud0Bb2gcJUsmBp8WdGRaIJclHnBu9E/fep6sierwSNjMgcyWtvQrb
ONwAmkYE1Y0ew5QLMN2hDsr9RmSO7mczGzNlcMM/IkUEp3jpaglvtaFcpWHxK5Vu
jLd9nppMlANySEaZGGQd5uYd9Imff+R8TCF8GwZJlu00+lbHuUtbwA0L7dl2aQCt
fBITC1KNYIU0P2VV1U++0DldeByLuGrtZDhIYpqf4sDF/SYxsI8ZeXDpP6sDEHiB
vEJ08Fc1wlmsijl9rBWZuXYigTPSbBFY1zE2PD2yt7dT4oUfexNmojM5jzJMF7HE
z3LnX4eZLrIg0JffrTdu0kjIVazIgsgzmA+kTClrno6Gwa9bc3LyxtPf2gb3f2gG
3sohTpmLgwFiBeGCxjoQ+QzBaFcAO1wvgub1UKl0ke5KWIT6MrxVJM5cLwQXAghv
fxwcLRK0g0fnYKBcY1Ezw+eZTGnxF3DHkVdy8CZy9pv+jSiBbKqTAeSaMQARAQAB
tDlGZXJuYW5kbyBBbGV4ZWkgUm9tZXJvIEdhc3BhciAobGFiMikgPDExMzIyNTU1
MjBAcnVkbi5ydT6JAlEEEwEKADsWIQQfOpnPjLX/rQGGKdr/SI1N2Lc89gUCancJ
iwIbAwULCQgHAgIiAgYVCgkICwIEFgIDAQIeBwIXgAAKCRD/SI1N2Lc89jS1EACf
EeG6GK2ET77SDB0VgXZXc2dnJiEt48sF6dW+LlcwBjCxFbkNuj6hpK6/SYddYfOP
tj7Cgj2CMlJMsDVJiWAUx5463tGRmlUN6dpFhfF4I6NtsRZnI//hoj2l2YSiXQAC
9dW/euKva0ue+FJHqjgA16KZAC0vFibZHQB/NYW4H+I4MTNAlLzGHLxj2bWbOULD
zWCDZa7sYE3ULXNq5a7d9vQuoNckKijo015yPan5rNCwLPPU2iOyPvgrQCCGoF1V
QwLzYLxxaY7+RFe/V8ah4QMefEtKPpSG0B9Quekluqrv0C+YnAMcwYg7yqRK7LwU
```

Рис. 14: Export the PGP key

Затем ключ был добавлен в веб-интерфейс GitHub:

- Войдите в GutHub.
- Перейдите в Настройки Ключи SSH и GPG.
- В разделе “Ключи GPG” нажмите кнопку “Новый ключ GPG”
- В поле “Заголовок” введите описательное имя:Fedora GPG.
- Нажмите “Добавить ключ GPG”.

7.7. Настройка автоматических подписей коммитов git

Чтовы гарантировать автоматическую подпись всех коммитов в репозитории,Git был настроен на использование ключа PGP,сгенерированного еа предыдущем шаге,без необходимости добавлять опцию ‘-S’ к каждому коммиту. Это гарантирует,что

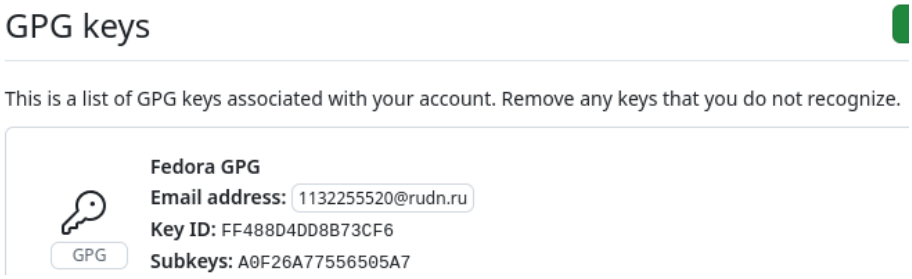


Рис. 15: PGP key in GitHub

ни один коммит не останется неподписанным им-за не недосмотра.

Преимущества автоматической подписи:

- Автоматизация:Нет неоюходимости помнить о добавлении ‘-S’ к каждому коммиту.
- Безопасность:Все коммиты подписываются автоматически.
- Соответствие требованиям: Обеспечивает соответствие политикам безопасности проекта.

Для включения автоматической подписи были выполнены следующие команды:

1. Настройка идентификатора ключа подписи:

```
liveuser@localhost-live:~$ git config --global user.signingkey FF488D4DD8B73CF6
```

Рис. 16: Use og user.signingkey

Эта команда указывает Git,какой ключ PGP использовать для подписи коммитов. Идентификатор (Key ID)-это тот,который получен на шаге 7.4 (Настройка ключей PGP).

2. Включите автоматическую подпись коммитов:

```
liveuser@localhost-live:~$ git config --global commit.gpgsign true
```

Рис. 17: use of commit.gpgsign

Сэтой конфигурацией все коммиты,созданные с этого момента,будут автоматически подписываться без необходимости использования опции -S.

3. Настройте переменную GPG_TTY для предотвращения ошибок:

На некоторых системах при подписании коммитов может появиться ошибка “пзп не удалось подписать данные”. Для исправления этой проблемы была настроена переменная среды GPG_TTY:

```
liveuser@localhost-live:~$ echo 'export GPG_TTY=$(tty)' >> ~/.bashrc
liveuser@localhost-live:~$ source ~/.bashrc
```

Рис. 18: export GPG_TTY

Проветка конфигурации:

Для проверки правильности примения конфигурации были выполнены следующие команды:

```
liveuser@localhost-live:~$ git config --get user.signingkey
FF488D4DD8B73CF6
liveuser@localhost-live:~$ git config --get commit.gpgsign
true
```

Рис. 19: comand to see the gpgsign and signingkey

В резултате должны отобразиться идентификатор ключа и значение true сщщтветственно.

7.8. Настройка gh

Аутентификация GitHub:

Для начала процесса аутентификации была выполнена следующая команда:

Прцесс проведет пользователя через ряд вопросов:

- В какую учетную запись вы хотите войти?:Был выбран GitHub.com.
- Какой протокол вы предпочитаете испольэловать для операций с Git?:Был выбран SHH (поскольку ключи SSH ьыли сгенерировары на шаге 7.3).
- Сгенерировать новый ключ SSH для добавления в ващу учетную запись GitHub?:Был выбран “Нет” (ключ SSH уже был сгенерирован).

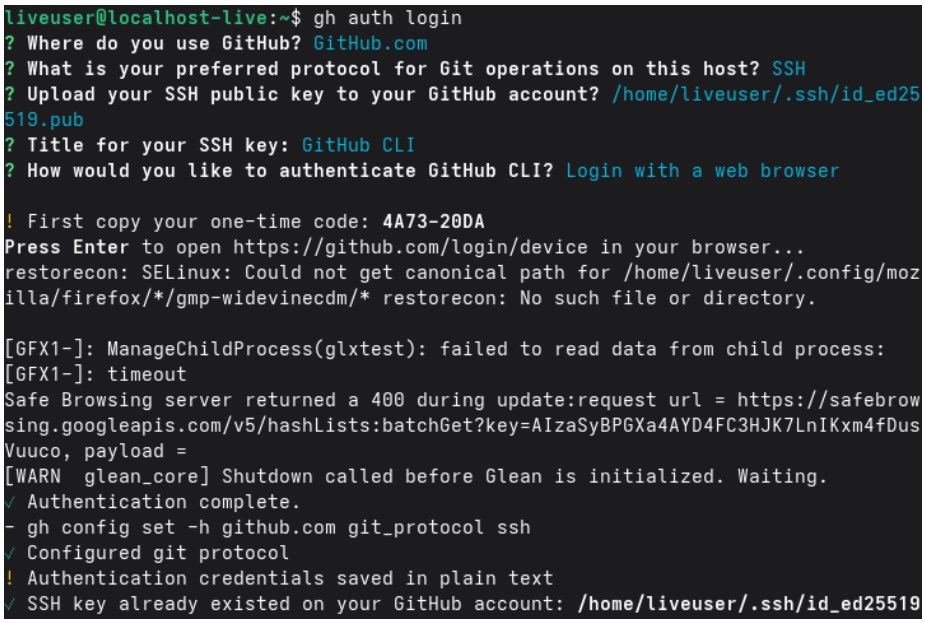


Рис. 20: use of the gh auth login

- Как вы хотите пройти аутентификацию?:Был выбран вход через веб-браузер.

Затем браузер открылся автоматически для завершения процесса аутентификации. Были введены учетные данные GitHub,и авторизация была подтверждена.

Проверка аутентификации:

Для проверки успешности аутентификации была выполнена следующая команда:

Ожидаемый результата должен отображать информацию об аутентифицированной учетной записи и используемом протоколе (HTTPS/SSH).

7.9. Шаблон для рабочего пространства

7.9.1. Создание репозитория курса на основе шаблона

для создания рапозитория курса были выполнены следующие шаги:

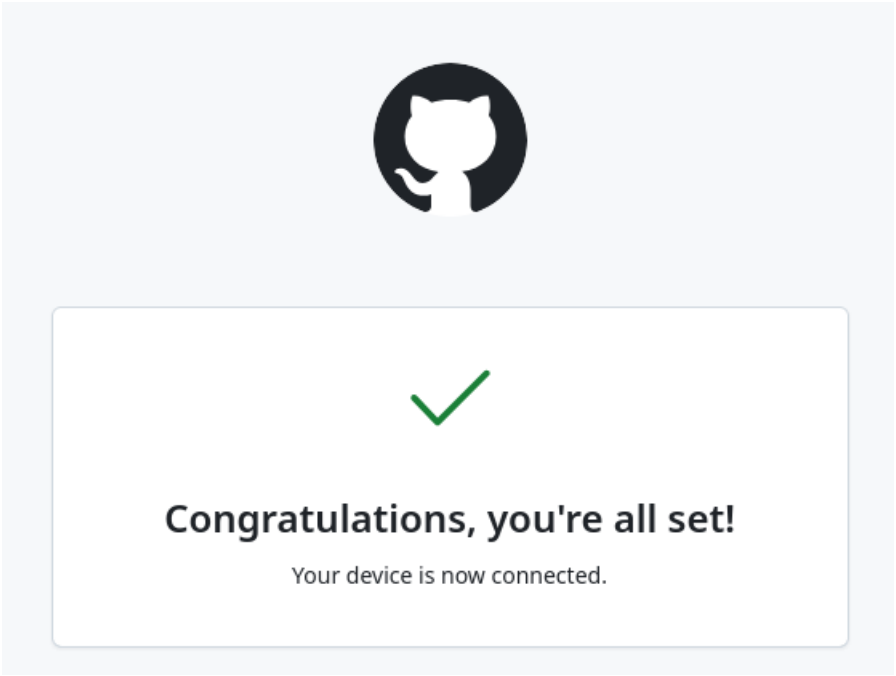


Рис. 21: Verification github

```
liveuser@localhost-live:~$ gh auth status
github.com
  ✓ Logged in to github.com account AlexeiFRG1 (/home/liveuser/.config/gh/hosts.
yml)
  - Active account: true
  - Git operations protocol: ssh
  - Token: gho_************************************
  - Token scopes: 'admin:public_key', 'gist', 'read:org', 'repo'
```

Рис. 22: auth status

Шаг 1: Создание структуры каталогов

Иерархия каталогов для курса была создана в каталоге ~/work/study/:

```
liveuser@localhost-live:~$ mkdir -p ~/work/study/2025-2026/"Архитектура компьюте
pa"
liveuser@localhost-live:~$ cd ~/work/study/2025-2026/^C
liveuser@localhost-live:~$ cd ~/work/study/2025-2026/"Архитектура компьютера"
```

Рис. 23: Create document

Шаг 2: Создание репозитория на основе шаблона

Для создания нового рапозитория на GitHub на основе предоставленного университетом шаблона использовалась команда gh repo create:

```
liveuser@localhost-live:~/work/study/2025-2026/Архитектура компьютера$ gh repo c
reate study_2025-2026_arch-pc --template=yamadharma/course-directory-student-tem
plate --public
GraphQL: Could not clone: Name already exists on this account (cloneTemplateRepo
sitory)
```

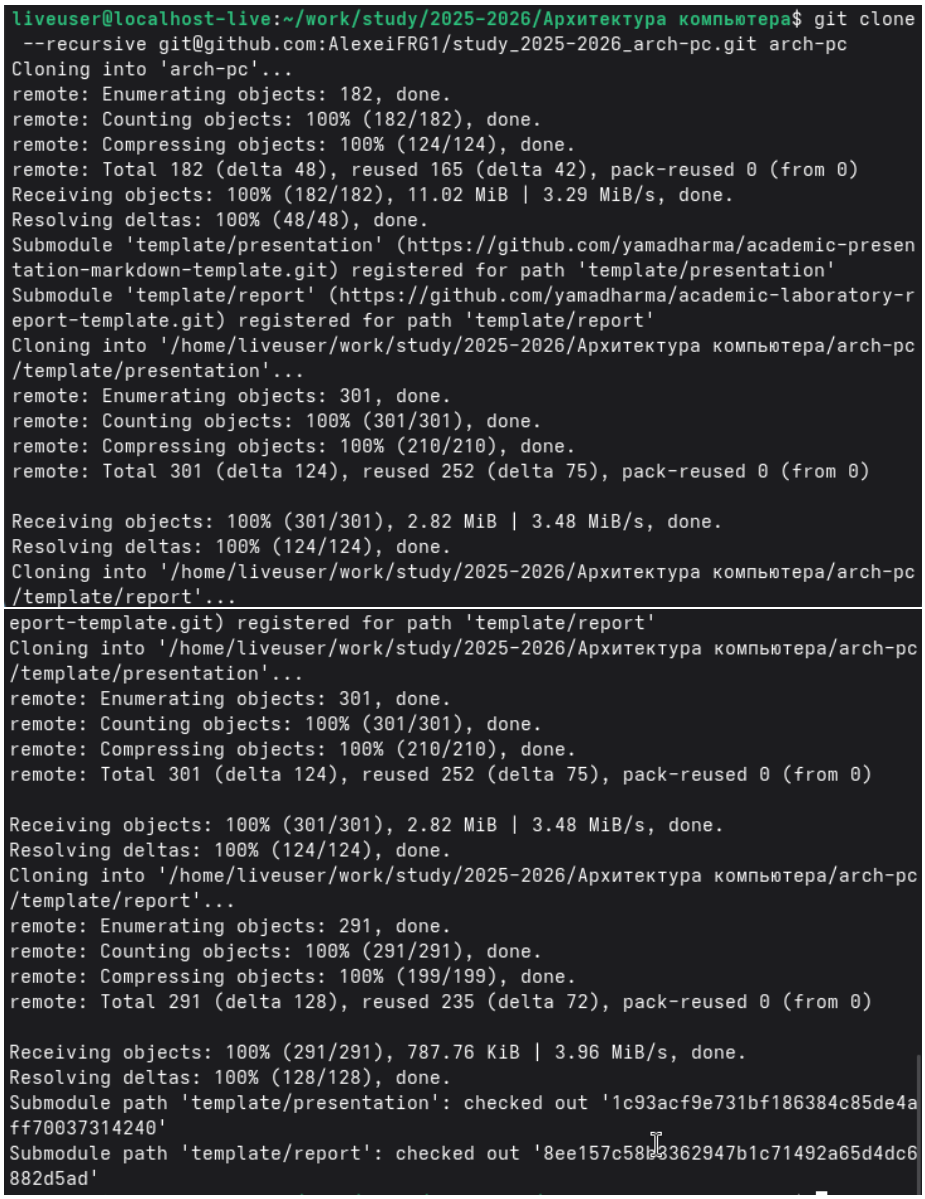
Рис. 24: repository create

Эта команда:

- Создает общедоступный репозиторий на GitHub с именем study_2025-2026_arch-pc.
- В качестве основы используется шаблон yamadharma/course-directory-student-template.
- Опиция ‘–public’ делает репозиторий видимым для всех.

Шаг 3:Клонируйте репозиторий на свой локальный компьютер

Для начала работы клонированный на локальный компьютер рапозиторий:



Настройка каталога курса

Шаг 1:Перейдите в каталог курса

Шаг 2:Создайте файл COURSE

Для указания имении курса был создан файл с именем COURSE:

```
liveuser@localhost-live:~$ cd ~/work/study/2025-2026/"Архитектура компьютера"/ar
ch-pc
```

Рис. 25: arch-pc

```
liveuser@localhost-live:~/work/study/2025-2026/Архитектура компьютера/arch-pc$ e
cho arch-pc > COURSE
```

Рис. 26: Course

Шаг 3:Сгенерируйте структуру курса

Для генерации структуры каталогов и файлов,необходимых для курса,была выполнена команда make:

```
liveuser@localhost-live:~/work/study/2025-2026/Архитектура компьютера/arch-pc$ m
ake
Usage:
  make <target>

Targets:
  list                          List of courses
  prepare                       Generate directories structure
  submodule                     Update submules
```

Рис. 27: use of make

После генерации структуры изменения были сохранены и отправлены в удаленный рапозиторий:

Проверка:

Чтобы убедиться в правильности создания структуры,вы можете вывести содержимое каталога:

Каталог должен содержать структуру папок,соответствующую курсу (лабораторные работы,отчеты и т.д.)

Контрольные вопросы

В этом разделе представлены ответы на контрольные вопросы,приверденные в конце лабораторной работы,которые позволяют понимание основных концепций систем контроля версий и Git/

```
liveuser@localhost-live:~/work/study/2025-2026/Архитектура компьютера/arch-pc$ g
it add .
liveuser@localhost-live:~/work/study/2025-2026/Архитектура компьютера/arch-pc$ g
it commit -m "make course structure"
On branch master
Your branch is up to date with 'origin/master'.

nothing to commit, working tree clean
liveuser@localhost-live:~/work/study/2025-2026/Архитектура компьютера/arch-pc$ g
it push
Everything up-to-date
```

Рис. 28: use of git add

```
liveuser@localhost-live:~/work/study/2025-2026/Архитектура компьютера/arch-pc$ l
s -la
total 76
drwxr-xr-x.  6 liveuser liveuser   340 Aug  8 11:55 .
drwxr-xr-x.  3 liveuser liveuser    80 Aug  8 12:00 ..
-rw-r--r--.  1 liveuser liveuser     8 Aug  8 12:03 COURSE
drwxr-xr-x.  8 liveuser liveuser   280 Aug  8 12:08 .git
-rw-r--r--.  1 liveuser liveuser  1765 Aug  8 11:55 .gitattributes
-rw-r--r--.  1 liveuser liveuser  4722 Aug  8 11:55 .gitignore
-rw-r--r--.  1 liveuser liveuser   278 Aug  8 11:55 .gitmodules
drwxr-xr-x. 13 liveuser liveuser   300 Aug  8 11:55 labs
-rw-r--r--.  1 liveuser liveuser 18657 Aug  8 11:55 LICENSE
-rw-r--r--.  1 liveuser liveuser  1021 Aug  8 11:55 Makefile
-rw-r--r--.  1 liveuser liveuser   348 Aug  8 11:55 package.json
-rw-r--r--.  1 liveuser liveuser     0 Aug  8 11:55 prepare
drwxr-xr-x.  4 liveuser liveuser   120 Aug  8 11:55 presentation
-rw-r--r--.  1 liveuser liveuser   152 Aug  8 11:55 README.en.md
-rw-r--r--.  1 liveuser liveuser  5653 Aug  8 11:55 README.git-flow.md
-rw-r--r--.  1 liveuser liveuser 12339 Aug  8 11:55 README.md
drwxr-xr-x.  5 liveuser liveuser   100 Aug  8 11:55 template
```

Рис. 29: see the list

1. Что такое системы контроля версий (VCS) и для решения каких задач они предназначаются?

Система контроля версий (VCS)-Это инструмент,который регистирует изменения в одном или несколких файлах с течением времени,позволяя восстанавливать предыдущие версии проекта. Они предназначены в первую очередь для:

- Управления историей изменений:Ведение полного журнала всех изменений,внесенных в исходный код.
- Облегчения совместной работы:Позволяет нескольким разработчикам одновременно работать над одним проектом,не перезаписывая работу друг друга.
- Отката изменений:Возможность возврата к предыдущим версиям кода в случае ошибок или проблем.
- Управления ветками (branches):Сщздание независимых линий разработки для экспериментов без влияния на стабильную версию проекта.

Разрешения конфликтов:Помощ в разрешении разлияий,когда два или болле разработчика изменяют одни и те же части кода.

2. Объясните следующие понятия VCS и их отношения:хранилище,commit,история,рабочая копия.

Хранилище(репозиторий):Это место,где хранятся все файлы проекта вместе с их историей изменений. Репозиторий может быть локальным (на компьютере разработчика) или удаленным (на сервере,например,GitHub).

Commit (Фиксация изменений):Это снимок (snapshot) состояния репозитория в определенный момент времени. Каждый commit содержит набор изменений,описательное сообщение,дату и время,а также информацию об авторе. Commit представляет собой атомарную единицу изменения в истории проекта.

История:Это хронологическая последовательность всех commit-ов,сделанных в репозитории. Она позволяет просматривать прошлое проекта,видеть,кто,когда и какие изменения вносил,а также при необходимости возвращаться к более ранним версиям.

Рабочая копия:Это текущая версия файлов проекта,которая находится на локальном компьютере разработчика. Именно над

этой копией выполняется работа:редактируются файлы,добавляются новые,удаляются старые. Рабочая копия связана с репозиторием,и изменения,внесенные в нее,могут быть зафиксированы (committed) для обновления репозитория.

Связь между понятиями:Разработчик работает со своей рабочей копией,внося изменения. Когда он считает,что изменения готовы,он группирует их в commit,который добавляется в историю репозитория. Таким образом,репозиторий всегда содержит последнюю версию кода и всю историю изменений.

3. То представляют собой и чем отличаются централизованные и децентрализованные VCS?Приведите примеры VCS каждого вида.

Централизованные VCS (CVCS):

В этой модели существует единый центральный репозиторий,расположенный на сервере,который хранит все версии проекта. Разработчики загружают рабочую копию файлов с этого центрального сервера и после внесения изменений отправляют их обратно в центральный репозиторий.

Преимущества:

- Простота администрирования (единая точка контроля).
- Централизованное управление доступом и првами.

Недостатки:

- Зависмость от центрального сервера:если сервер выходит из строя,работа останавливается.
- Невозможность работать без подключения к интернету для обмена изменениями.

Примеры:CVS,Subversion(SVN).

Распределенные VCS (DVCS):

В DVCS каждый разработчик имеет полную копию репозитория на своем локальном компьютере,включая всю историю версий. Это устраняет зависимость от единственного централбного сервера,поскольку каждая локальная копия функционирует как полноценный репозиторий.

Преимущества:

- Возможность работы без подключения к интернету.
- Повышенная устойчивость (лобая копия может восстановить проект).
- Ветки (branches) более легкие и простые в управлении.

Недостатки:

- Более высокая начальная сложность.
- Большее использование дискового пространства (каждая копия содержит всю историю)

Примеры:Git,Mercurial,Bazaar.

4. Опишите действия с VCS при единоличной работе с хранилищем.

При индивидуальной работе с репозиторием типичный рабочий процесс выглядит следующим образом:

- Инициализация репозитория:

```
mkdir проект
cd проект
git init
```

- Создание или изменение файлов:Работа над кодом или файлами проекта.
- Проверка состояния:

```
git status
```

- Добавление изменений в область подготовки (staging area):

```
git add. #Добавляет все изменения
 #или
git add имя_файла #Добавляет конкретный файл
```

- Фиксация изменений (commit):

```
git commit -m "Описательное сообщение об изменении"
```

- Просмотр истории:

```
git log
```

- Откат изменений при необходимости: Для отмены неподтвержденных изменений:git checkout –Файл Для возврата к предыдущему commit:git revert

5. Опишите порядок работы с общим хранилищем VCS.

При работе с общим (коллаборативным) репозиторием,имеющим центральный репозиторий,типичный рабочий процесс выглядит следующим образом:

- Получение последних изменений из удаленного репозитория:

```
git pull
```

- Создание ветки для работы над новой функциональностью (рекомендуется)

```
git checkout -b имя_ветки
```

- Внесение изменений в код
- Проверка состояния изменений:

```
git status
```

- Добавление изменений в область подготовки:

```
git add.
```

- Фикчация изменений с описательным сообщением:

```
git commit -m "Описание изменения"
```

- Отправка изменений в удаленный репозиторий:

```
git push origin имя_ветки
```

- Если работа ведется в ветке,сляние с основной веткой:

```
git checkout master
git merge имя_ветки
git push origin master
```

- Разрешение конфликтов при их возникновении (когда два человека изменяют одни и те же строки файла).

6. Каковы основные задачи,решаемые инструментальным средством git?

Git решает несколько фундаментальных задач в разработке программного обеспечения:

- Распределенное управление версиями:Позволяет каждому разработчику иметь полную копию репозитория,что облегчает работу без подключения к сети и совместную работу.
- Управление ветками (branches):Предоставляет очень эффективную и легкую систему ветвления,позволяющую работать одновременно над несколькими линиями разработки.
- Целостность данных:Использует хэши SHA-1 для идентификации каждого commit,гарантируя,что данные не будут повреждены.
- Эффективность производительности:Быстрый даже для проектов большого размера.
- Полная история:Ведет детальный журнал всех изменений,позволяя проводить аудит и откаты.
- Совместная работа:Облегчает работу в команде благодаря интеграции с платформами таким как GitHub,GitLab или Bitbucket.

7. Назовите и дайте краткую характеристику командам git.

- git init:Инициализирует новый репозиторий Git в текущем каталоге.
- git add:Добавляет файлы в область подготовки (staging area) для следующего commit.
- git commit:Сохраняет изменения из области подготовки в локальном репозитории с сообщением.

- git status:Показывает текущее состояне репозитория (измененные,добавленные файлы и т.д.)
- git diff:Показывает различия между текущим состоянием и последним commit.
- git log:Показывает историю commit-ов репозитория.
- git branch:Выводит список,создает или удаляет ветки.
- git checkout:Переключается между ветками или восстанавливает файлы.
- git merge:Сливает изменения из одной ветки в другую.
- git clone:Создает локальную копию удаленного репозитория.
- git pull:Получает изменения из удаленного репозитория и сливает их с локальной работой.
- git push:Отправляет локальные изменения в удаленный репозиторий.
- git remote:Управляет удаленными репозиториями,связанными с локальным репозиторием.

8. Приведите примеры использования при работе с локальным и удалённым репозиториямию

С локальным репозиторием:

1. Инициализация локального репозитория:

```
mkdir мой_проект
cd мой_проект
git init
```

2. Создание файла и добавление его репозиторий:

```
echo "Привет мир" > файл.txt
git add файл.txt
git commit -m "Первый commit"
```

3. Посмотр истории:

```
git log
```

С удаленным репозиторием:

1. Клонирование удаленного репозиторя:

```
git clone https://github.com/пользователь/репозиторий.git
```

2. Добавление уданного репозитория к существующему локльному:

```
git remote add origin https://github.com/пользователь/репозиторий.git
```

3. Отправка изменений в удаленного репозиторий:

```
git push -u origin master
```

4. Получение изменений из удаленного репозитория:

```
git pull
```

9. Что такое и зачем могут быть нужны ветви (branches)?

Ветки (branches)-Это независмые линии разработки,которые позволяют работат над различными функциональными возможностями или экспериментами,не затрагивая основную (стабильную) версию проекта.

Зачем они нужны:

- Параллельная разработка:Позволяют нескольким разработчикам одновременно работать над различными функциональностями.
- Эксеперименты:Позволяют тестировать новые идеи без риска сломать стабильный код.
- Организация работы:Помагают структурировать разработку вокруг версий,функциональностей или исправлений ошибок.
- Слияние (Merge):После завершения работы в ветке ее можно слить с основной веткой,интегрируя изменения контролируемым образом.

10. Как и зачем можно игнорировать некоторые файлы при commit?

Файлы игнорируются путем создания файла .gitignort в корне репозитория,в котором указываются шаблоны имен файлов или каталогов,которы Git должен игнорировать.

Зачем игнорировать файлы:

- Избежание ненужных файлов: Временые файлы,логи,файлы сборкм м бинарные файлы не должны включаться в репозиторий.
- Уменьшение размера репозитория: Поддержание репозитория легким и эффективным.
- Безопасность:Предотвращение загрузки файлов с конфиденциальной информацией (пароли,ключи API).
- Поддержание чистоты: Предотврщение зарянения истории репозитория автоматически генерирумыми файлами.

#выводы

В ходе выполнения лавораторной работы я освоил базовые навыки работы с Git: установил и настроил систему,создал SSH-ключи и PGP-ключи для безопасного подключения и подписи коммитов,а также настроил автоматическую подпись. я зарегистрировался на GitHub,созлад репозиторий на основе шаблона и настроил рабочее пространство для дальнейших работ. Все поставленные задачи выполнены,полученные навыки являются основой для дальнейшего использования систем контроля версий.

#Список литературы

- Защищенныйю(s.f.).Системы контроля версий из
- Мастер приложений. (2020).Контроль версий из
- Университет Вальядолида. (2018). Менеджеры контроля версий. В TFG-I-1266 из
- Arsys. (2024). Что такое системы контроля версий? из
- Microsoft Learn. (2024). Совместная работа над кодом-репозитории Azure из
- Чакон,С.,и Штрауб,Б.(2014). Pro Git (2-у изд.). Нажмите. из
- Атлассиан. (2014). Руководство по GitHub по SSH. из
- Документы GitHub.(s.f.). Подключению к GitHub по SSH. из
- Red Hat. (2023). Руководство по администрированию SSH из
- OptnSSH. (2024). Инструкция по удалению ssh-кейгена из
- Чакон,С.,и Штрауб.Б.(2014). Руководство (2-у изд.).Приложение из
- Цифоровой окен.(2024).Как настроить SSH-ключи в Linux из
- Документы на GitHub.(С.Ф.)Acerca de la verificacion firma из

- Докуметы на GitHub. (С.ф.).Генерировать и читать паревода УНА Нуэва Клаве ГОБю из
- Документы на Гитхабе. (s.f.).Я открыл доступ к ПЗП как источнику информации на Гитхабе из
- Башня мерзавцев. (2025). Как (и зачем) подписывать коммиты с помощью GPG из
- Документы GitHub/ (s.f.). Проверке подписи коммитов из